

Sistemas Distribuídos — Semana 04

Sincronismo II e Paradigma Cliente/Servidor · Aulas 7 (25/08) e 8 (27/08) · Prof. Leandro Borges

Exclusão mútua distribuída

Sem memória compartilhada, garantir que apenas um processo acesse um recurso crítico por vez exige coordenação explícita pela rede — o problema clássico de exclusão mútua ganha uma nova dimensão em sistemas distribuídos.

Existem três abordagens clássicas. A centralizada usa um coordenador único que concede permissão de acesso a quem solicitar: é simples de implementar, mas cria um ponto único de falha — se o coordenador cair, ninguém mais consegue acessar o recurso. A distribuída, exemplificada pelo algoritmo de Ricart-Agrawala, faz os processos trocarem mensagens de pedido e permissão entre si diretamente, usando timestamps (como os de Lamport, vistos na aula anterior) para decidir quem tem prioridade quando há disputa. Já a abordagem em anel faz circular um token entre os processos organizados logicamente em círculo — só quem possui o token naquele momento pode acessar o recurso crítico.

Posicionamento global de nós

Saber a localização — lógica ou física — dos nós de um sistema distribuído é importante por vários motivos práticos: ajuda a rotear mensagens de forma eficiente, permite balancear carga entre servidores e ajuda a decidir onde executar uma determinada tarefa. Sistemas geo-distribuídos, por exemplo, usam essa informação para reduzir a latência percebida pelo usuário, direcionando cada requisição para o servidor fisicamente mais próximo dele.

Algoritmos de eleição

Quando o coordenador de um grupo de processos falha, o sistema precisa eleger um novo coordenador de forma automática, sem intervenção humana. O Algoritmo do Valentão (Bully) resolve isso elegendo o processo de maior identificador que ainda está ativo: qualquer processo que perceba a falha do coordenador atual pode iniciar uma eleição, desafiando os processos de identificador maior até que o de maior id responda e assuma o papel. Já o Algoritmo em Anel organiza os processos em um círculo lógico, fazendo uma mensagem de eleição circular até que todos tenham visto o maior identificador do grupo, que então se torna o novo coordenador.

Paradigma cliente/servidor

Depois de tratar coordenação entre processos, voltamos a algo mais concreto: o modelo mais usado na prática para construir sistemas distribuídos. Nele, o servidor espera conexões, processa as requisições recebidas e devolve respostas, geralmente atendendo a múltiplos clientes ao mesmo tempo. O cliente, por sua vez, inicia a comunicação, envia requisições e consome as respostas do servidor. Para que essa troca funcione, ambos precisam concordar previamente em um protocolo de comunicação — pode ser um protocolo já estabelecido, como o HTTP, ou um protocolo próprio da aplicação construído sobre TCP.

Sockets: fundamentos práticos

Na prática, a comunicação cliente/servidor mais básica é implementada usando sockets, e o fluxo típico do lado do servidor segue quatro passos. Primeiro, cria-se o socket e o vincula (bind) a um endereço IP e uma porta específica. Em seguida, o servidor entra em modo de escuta (listen), aguardando que clientes se conectem. Quando uma conexão chega, o servidor a aceita (accept), o que cria um socket dedicado a essa conexão específica, separado do socket original que continua escutando por novas conexões. Por fim, cliente e servidor trocam dados livremente através de operações de envio e recebimento (send/receive), até que a comunicação termine e as conexões sejam encerradas.

Referências

COULOURIS, G. et al. Sistemas Distribuídos: Conceitos e Projeto. 5. ed. Porto Alegre: Bookman.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.